

# Classes

**Luca Parisi, EPCC**

[l.parisi@epcc.ed.ac.uk](mailto:l.parisi@epcc.ed.ac.uk)

# User defined types

Called "class types" formally - can be defined with either the `class` or `struct` keyword.

```
struct Complex {  
    double re;  
    double im;  
};
```

Let's show this on Compiler Explorer: <https://godbolt.org/z/d3MfGhPz5>

This style of type is often called a "plain old data" (POD) type or a "trivial" type.

This simply aggregates together several *members* which have a name and type. You can then pass them around together and access the data.

# Class types

Creating trivial types - give the class name then list the values to be assigned to the members, *in order*, inside braces:

```
Complex mk_imaginary_unit() {  
    return Complex{0, 1};  
}
```

This is called aggregate initialisation.

Alternatively:

```
Complex mk_imaginary_unit() {  
    Complex sqrt_m1; // Values are uninitialised  
    sqrt_m1.re = 0;  
    sqrt_m1.im = 1;  
    return sqrt_m1;  
}
```

# Class types

Using trivial types:

```
void test() {  
    Complex z = mk_imaginary_unit();  
    assert(z.re == 0);  
    assert(z.im == 1);  
    return 0;  
}
```

Any piece of code can access the member variables inside the object.

# Creating instances

# Default member initialisers

In the class definition you can (should?) provide default values for member variables:

```
struct Complex {  
    double re = 0.0;  
    double im = 0.0;  
};
```

Now when you create an object it will be initialised to a known state for you:

```
void test() {  
    Complex z;  
    assert(z.re == 0);  
    assert(z.im == 0);  
}
```

# Default member initialisers

A note for C++11: you lose the ability to set the member variables by just providing the values:

```
struct Complex {  
    double re = 0.0;  
    double im = 0.0;  
};  
  
Complex mk_imaginary_unit() {  
    return Complex{0, 1};  
}
```

GCC says:

```
error: no matching constructor for initialization of 'Complex'  
    return Complex{0,1};
```

# Constructors

Often you want to control the creation of instances of your classes.

You do this with *constructors* - these are special member "functions" with the same name as the type.

```
struct Complex {  
    Complex() = default;  
    Complex(double re);  
    Complex(double re, double im);  
    double re = 0.0;  
    double im = 0.0;  
};
```

Note we declare three:

- one that initialises with a purely real value
- one that initialises with a real and imaginary value
- a *default constructor* which needs no arguments (that we tell the compiler to generate for us as before with `= default` )

# Constructors

- Constructors are not directly callable
- Constructors do not return a value
- Constructors can do initialisation of member variables before the body begins execution

Let's define the ones we declared just now:

```
Complex::Complex(double real) : re{real} {  
}  
  
Complex::Complex(double real, double imag) : re{real}, im{imag} {  
}
```

# Creating complexes

We can now create complex numbers different ways

```
auto zero = Complex{}; // Uses default member initialisers
auto pi = Complex{3.14159}; // pi.im == 0.0
auto sqrt_minus_one = Complex{0, 1};
Complex one = 1.0;
```

# Member functions

# Object-oriented programming

Object oriented programming is one of the major paradigms supported by C++

OOP is based on the concept of "objects", which may contain data and code. A feature of objects is that an object's procedures can access and often modify the data of the object with which they are associated.

In C++ these attached bits of code are known as member functions - other languages might call them methods. For example:

```
int main() {  
    std::string name;  
    std::cin >> name;  
    std::cout << "Hello, " << name << ". "  
               << "Your name has " << name.size() << " characters." << std::endl;  
    return 0;  
}
```

# Member functions

Typically these are *declared* in the class definition...

```
// complex.hpp
struct Complex {
    // Constructors as before
    double magnitude() const;

    double re = 0.0;
    double im = 0.0;
};
```

# Member functions

... and *defined* out of line

```
// complex.cpp
double Complex::magnitude() const {
    return std::sqrt(re*re + im*im);
}
```

Within a member function we can access the members (data or function) by just giving their names.

Outside the class, we need to have an instance to use:

```
Complex z{3, 4};
assert(z.magnitude() == 5.0);
```

# Classes and structs

You define a class using either the `struct` or `class` keywords - technically the difference is only the default *accessibility* of members.

Members can be:

- `public`: usable from any piece of code - i.e. the public interface of the class - default for `struct`
- `private`: only usable from within the context of the class - i.e. an implementation detail of the class that should be of no interest to code that uses it - default for `class`

Restricting access to implementation details is part of encapsulation: another common aspect of OOP

# Access Control

```
class Greeter {  
    std::string greetee;  
    void say_hello() const {  
        std::cout << "Hello, " << greetee << std::endl;  
    }  
};  
  
void test(Greeter const& g) {  
    g.say_hello();  
}
```

# Access Control

```
class Greeter {  
    std::string greetee;  
public:  
    void say_hello() const {  
        std::cout << "Hello, " << greetee << std::endl;  
    }  
};  
  
void test(Greeter const & g) {  
    g.say_hello();  
}
```

# C++ compilation

# Declarations vs Definitions

C++ distinguishes between *declaration* and *definition*.

A **declaration** tells the compiler that a name exists, what kind of entity it refers to and (if it is a function or variable) its type. For most uses this is all the compiler needs. Declarations can be repeated, as long as they match *exactly*.

A **definition** tells the compiler everything it needs to create something in its entirety. A definition is also a declaration. The one-definition rule says that definitions must not be repeated (with an exception).

# Where to put these

- Conventionally, one puts declarations of functions, definitions of classes, and global constants in **header** files.
  - Common suffixes are: .hpp, .h, .H
- Definitions of most functions should be in **implementation** files
  - Common suffixes are .cpp, .cxx, .cc, .C
- Headers can be be `#include` into other files that need to use the types and function declared there.

# Where to put these

E.g. Complex.hpp:

```
#ifndef COMPLEX_HPP
#define COMPLEX_HPP

struct Complex {
    Complex() = default;
    Complex(double real, double imag);
    double real() // etc...
private:
    double re;
    double im;
};

#endif
```

Complex.cpp

```
#include "complex.hpp"

Complex::Complex(double real, double image) : re(real), im(imag) {}

double Complex::real() {
    return re
}
```

# Exercise

In your clone of this repository, find the `2.1-class-types` exercise and list the files

```
$ cd archer2-cpp/exercises/2.1-class-types
$ ls
Makefile  README.md  complex.cpp  complex.hpp  test.cpp
```

The files `complex.hpp` and `complex.cpp` contain the beginnings of a complex number class. Follow the instructions in the comments to complete the missing declarations in `complex.hpp` and then add the required out of line definitions in `complex.cpp`.

To test your implementation, `test.cpp` holds some basic unit tests created using the Catch2 unit testing library.

You can compile and run with:

```
$ make && ./test
g++ --std=c++14 -I../include -c -o complex.o complex.cpp
g++ complex.o test.o -o test
=====
All tests passed (36 assertions in 5 test cases)
```

# Operators

# Operators are functions

C++ operators, for the non-fundamental types, are just functions with odd names, e.g.:

```
std::string operator+(const std::string& a, const std::string& b);
```

You can then use the natural syntax when manipulating these in other code:

```
std::string user_name = "alice";  
auto data_file = user_name + ".csv";
```

# Operator overloading

Complex numbers have the usual arithmetic operations:  $+$   $-$   $\times$   $\div$

We can provide operator overloads, like:

```
struct Complex {  
    // Other members...  
  
    Complex& operator+=(Complex const& increment) {  
        re += increment.re;  
        im += increment.im;  
        return *this;  
    }  
};  
Complex operator+(Complex const& a, Complex const& b) {  
    return Complex{a.re+b.re, a.im+b.im};  
}
```

Here, `operator+=` is using the implicit `this` pointer

# Operator overloading

Complex numbers have the usual arithmetic operations ( $+$   $-$   $\times$   $\div$ ) and comparisons.

We can now use the natural syntax to add these values

```
Complex i{0, 1};  
  
Complex z;  
z += i;  
  
assert(z.re == 0 && z.im == 1);  
  
auto c = z + i;  
assert(c == 2*i);
```

# Constants

# Const-ness

Variables can be qualified with the `const` keyword

- objects marked with `const` cannot be modified
- compiler will give errors if you try

```
int main() {  
    int const i = 42;  
    std::cout << i << std::endl;  
    // prints 42  
  
    i += 10;  
    // error: cannot assign to variable 'i' with const-qualified type 'const int'  
    return 0;  
}
```

You should be adding `const` to your local variables whenever possible.

# Const-ness

Since function parameters are just local variables, they can be `const` too.

```
Complex operator-(Complex const a, Complex const b) {  
    return Complex{a.re - b.re, a.im - b.im};  
}
```

Since this function has no need to modify it's arguments, they should be marked as `const`

# Const-ness

Member functions have an implicit argument of the instance they "belong" to.

If they do not need to change an instance, then they should be marked `const` also:

```
struct Complex {  
    double magnitude() const;  
};
```

# East-const vs West-const

As far as the compiler and language specification are concerned, both of these are identical:

```
const int i = 42;
```

Rule: `const` applies to what is on its left, unless there is nothing on its left, then it applies to what is on its right

```
int const i = 42;
```

Rule: Always put `const` on the right of what it modifies

It doesn't really matter as long as you're **consistent** within a project.

# References

# Copying

By default, if you assign a variable a new value, C++ will copy it

```
double copy = original;
```

Same if you pass a value of some type to a function, it will be copied to a new local variable for the invocation of a function:

```
SimState AdvanceOneTimeStep(SimState old);  
  
// later...  
next_state = AdvanceOneTimeStep(current_state);
```

# References

C++ has the concept of a **reference**.

- A reference is a name that aliases an already existing object
- A reference must be initialised (function arguments are initialised by calling)
- A reference cannot be rebound to a new object

```
void ScaleVector(double scale, std::vector<double>& x) {  
    // Multiply every element of x by scale  
}  
  
void test() {  
    std::vector<double> data = ReadLargeFile();  
    ScaleVector(-1.0, data);  
}
```

# References and Const

You can qualify a reference as const:

```
SimState AdvanceOneTimeStep(SimState const& old) {  
    // do some science...  
    return new_state;  
}
```

# Functions can return a reference

Many classes have data members that they want to let their client have some access to without (necessarily) copying

Or access to a single value from the many they might contain

```
class AtomList {
    std::vector<Vec3> position;
    std::vector<Vec3> velocity;
    std::vector<double> charge;
public:
    std::vector<double> const& GetCharge() const {
        return charge;
    }
};

void analyse_md_data() {
    AtomList const atoms = ReadFromFile();
    std::vector<double> const& charges = atoms.GetCharge();
    std::cout << "Average charge = "
                << std::accumulate(charges.begin(), charges.end()) / charges.size()
                << std::endl;
}
```

# References and const with auto

When you declare a variable with auto you can qualify it as a reference and/or const:

- Use `auto x` when you want a modifiable copy
- Use `auto&& x` when you want a reference and the same const-ness as the initialiser
- Use `auto const &x` when you want a reference to original item and will not modify it

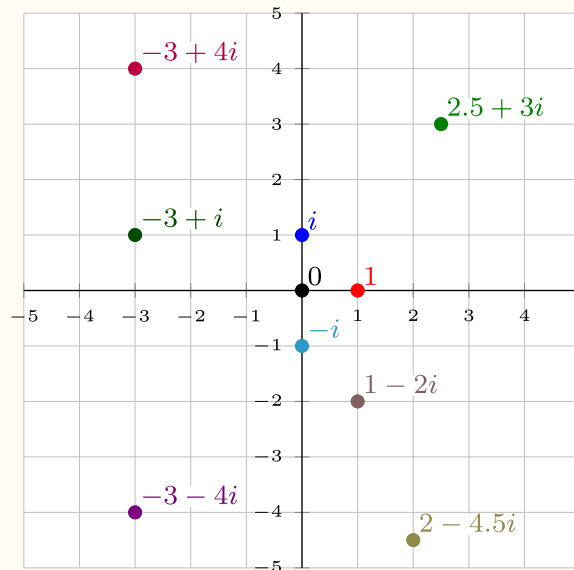


Use the last whenever possible

# Exercise

# Note: Complex numbers

- A complex number is a number that can be expressed in the form  $a+bi$  where:
  - $a$  is the real part
  - $b$  is the imaginary part
  - $i$  is the imaginary unit defined as the square root of  $-1$
- We can define basic operations for two complex numbers  $a=x+yi$  and  $b=u+vi$  as:
  - $a + b = (x + yi) + (u + vi) = (x + u) + (y + v)i$
  - $a - b = (x + yi) - (u + vi) = (x - u) + (y - v)i$



# Exercise

In your clone of this repository, find the 2.2-complex exercise and list the files

```
$ cd archer2-cpp/exercises/2.2-complex
$ ls
Makefile      complex.cpp   complex.hpp   test.cpp
```

The files `complex.hpp` and `complex.cpp` contain a partially working complex number class and `test.cpp` holds some basic unit tests.

You can compile and run with:

```
$ make && ./test
g++ --std=c++14 -I../include -c -o complex.o complex.cpp
g++ --std=c++14 -I../include -c -o test.o test.cpp
g++ complex.o test.o -o test
=====
All tests passed (34 assertions in 6 test cases)
```

But to get to this point you need to complete the code and fix a few bugs!